

Iteratori in Python

Iteratori in Python

Come funzionano gli iteratori in Python.

Perché ti serve conoscere gli iteratori

Ogni volta che scrivi `for elemento in lista`, Python sta usando un meccanismo nascosto chiamato **iteratore**. Capire come funziona ti permette di creare le tue sequenze personalizzate — anche sequenze che non finiscono mai, o che generano i dati "al volo" senza occupare memoria.

La differenza tra iterabile e iteratore

- **Iterabile**: qualcosa su cui puoi fare un ciclo `for`. Liste, stringhe, tuple, dizionari sono iterabili.
- **Iteratore**: un oggetto che "sa dove si trova" nella sequenza e sa come ottenere l'elemento successivo.

Pensa a un libro:

- Il **libro** è l'iterabile (contiene tutti i capitoli)
 - Il **segnalibro** è l'iteratore (ricorda a quale pagina sei arrivato)
-

Come funziona `for` dentro Python

Quando scrivi:

```
for x in [1, 2, 3]:  
    print(x)
```

Python esegue in realtà questi passi:

1. Chiama `iter([1, 2, 3])` per creare un iteratore dalla lista
2. Chiama ripetutamente `next(iteratore)` per ottenere un elemento alla volta
3. Quando la lista è finita, `next()` lancia `StopIteration` e il ciclo si ferma

Puoi vedere questi passi manualmente:

```
lista = [1, 2, 3]
iteratore = iter(lista)    # Crea il "segnalibro"

print(next(iteratore))     # 1 ← prende il primo elemento e avanza
print(next(iteratore))     # 2 ← prende il secondo elemento e avanza
print(next(iteratore))     # 3 ← prende il terzo elemento e avanza
# print(next(iteratore))   # Qui lancerebbe StopIteration: la lista è
                           # finita!
```

Creare un iteratore personalizzato

Puoi creare una classe che si comporta come un iteratore implementando due metodi speciali:

- `__iter__`: restituisce l'iteratore stesso
- `__next__`: restituisce il valore successivo (o lancia `StopIteration` se è finita)

```
class ContaDa:
    """Un iteratore che conta da 'inizio' fino a 'fine', incluso."""

    def __init__(self, inizio, fine):
        self.corrente = inizio    # Da dove partiamo
        self.fine = fine         # Dove ci fermiamo

    def __iter__(self):
        return self              # L'iteratore restituisce se stesso

    def __next__(self):
        if self.corrente > self.fine:
            raise StopIteration  # Abbiamo finito!
        valore = self.corrente
        self.corrente += 1       # Avanza al prossimo
        return valore

# Ora possiamo usarlo in un ciclo for, esattamente come una lista
for numero in ContaDa(1, 5):
    print(numero)

# → 1
# → 2
# → 3
# → 4
# → 5
```

Generatori: il modo più semplice

Creare una classe iteratore completa funziona, ma è verboso. I **generatori** ti permettono di fare la stessa cosa con molto meno codice.

Un generatore è una funzione che usa `yield` invece di `return`. Ogni volta che viene chiamata, si "ferma" a `yield`, restituisce il valore, e riprende da lì alla chiamata successiva — come un libro che segna automaticamente la pagina.

```
def conta_da(inizio, fine):
    corrente = inizio
    while corrente <= fine:
        yield corrente    # "Ecco il valore, mettiti in pausa finché non ti
        serve il prossimo"
        corrente += 1

# Si usa esattamente come prima
for numero in conta_da(1, 5):
    print(numero)
# → 1, 2, 3, 4, 5
```

La differenza rispetto a `return` : con `return` la funzione finisce e dimentica tutto. Con `yield` la funzione si mette in pausa e ricorda esattamente dove si trovava.

Generator expression: la versione compatta

Come le **list comprehension** (la sintassi `[x**2 for x in range(10)]` che crea una lista in una riga) ma con le parentesi tonde invece di quelle quadre. Producono un generatore invece di una lista: i valori vengono calcolati uno alla volta, solo quando servono, risparmiando memoria.

```
# List comprehension: crea TUTTI i quadrati in memoria subito
quadrati_lista = [x ** 2 for x in range(10)]

# Generator expression: produce UN quadrato alla volta, solo quando serve
quadrati_gen = (x ** 2 for x in range(10))

for q in quadrati_gen:
    print(q)    # → 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
```

Perché usare i generatori? Il problema della memoria

Immagina di dover lavorare con un milione di numeri:

```
# Senza generatore: tutti i numeri vengono creati in memoria subito
# Con 1.000.000 numeri, questo occupa decine di megabyte di RAM
numeri_lista = list(range(1_000_000))

# Con generatore: i numeri vengono creati uno alla volta, solo quando servono
# Occupa quasi zero memoria – indipendentemente da quanti numeri ci sono
numeri_gen = range(1_000_000)
```

I generatori sono come un nastro trasportatore: producono un pezzo alla volta, invece di mettere tutto sul tavolo in una volta.

Strumenti pronti: il modulo **itertools**

Python include una raccolta di iteratori già pronti, molto utili:

```
# Ciclo infinito su una sequenza (utile ad esempio per semafori o animazioni)
colori = itertools.cycle(["rosso", "giallo", "verde"])
for i in range(6):
    print(next(colori))    # → rosso, giallo, verde, rosso, giallo, verde

# Unire più sequenze in una sola
for x in itertools.chain([1, 2], [3, 4], [5]):
    print(x)    # → 1, 2, 3, 4, 5

# Prendere solo i primi n elementi di una sequenza (anche infinita)
for x in itertools.islice(range(1_000_000), 5):
    print(x)    # → 0, 1, 2, 3, 4
```